RBCCPS Server: Comprehensive User Manual

Available Compute Infrastructure

The RBCCPS compute server provides resources for research and project workloads. It consists of:

1. **Single Node** – functions as **both master (job scheduling) and compute node**.

Hardware Information

- **CPU:** Intel® Xeon® Platinum 8480+ (224 cores, 2 sockets, 2 threads per core)
- **GPU:** AMD Instinct MI300X
- **Memory:** 2.0 TiB
- **Storage:** 14 TB RAID5 (/mnt/data), 200 GB per user for personal storage
- **OS:** Ubuntu 22.04 LTS

Job Scheduling

The server uses the **SLURM Job Scheduler** to manage computational workloads.

Queue Name	Max Runtime
debug	24 hours

Terms and Conditions for Server Usage

To ensure the smooth operation and security of the RBCCPS server, users are required to adhereto the following terms and conditions:

Access Control: Access to the server is restricted to authorized users with valid credentials. Users are responsible for keeping their login credentials confidential and must not share them with others. Any unauthorized access attempts will be logged and reported.

1. **Responsible Usage:** Computing resources should be used efficiently, avoiding idle resource allocation. Personal, illegal, or unethical use of the server is strictly prohibited.
2. **Job Submission:** All jobs must be submitted through the SLURM job scheduler. Users must select the appropriate queue (as mentioned above). Jobs exceeding the time limits or resource quotas may be terminated without notice.
3. **Data Management:** Users are responsible for backing up their data. The server is not liable for data loss. Temporary files and unused data should be cleaned up promptly to free up storage. Sensitive or confidential data must not be stored or processed on the server without explicit permission.
4. **Monitoring:** The server administrator reserves the right to monitor and log all activities on the server for security and performance purposes. Logs will be reviewed periodically, and violations will result in suspension or termination of access.
5. **Maintenance:** Scheduled maintenance will be communicated in advance. Users should plan their jobs accordingly. Users must report any issues or irregularities to **computemanager.cps@iisc.ac.in**.

By using the RBCCPS server, users agree to comply with these terms and conditions. Failure to adhere to these guidelines may result in the loss of server privileges

Deletion of Account

User accounts that remain inactive for 90 days will be temporarily blocked. If the account remains inactive for an additional 90 days, the user account and data will be permanently deleted without prior notification.

The duration specified in the resource request form will be considered the usage deadline. If additional server access is needed beyond this period, users must provide prior notification.

Instructions on How to Access and Use the Server

Create an Account

Kindly use the form below to request the creation of an account on the RBCCPS cluster: Resource Request Form

After submitting the form, please wait for a confirmation email from the office (computemanager.cps@iisc.ac.in) regarding the creation of the account. If the information provided in the form is complete, the account will be created, and you will receive the confirmation email by the end of the next working day.

Login to the Server

To access the cluster, connect to the RBCCPS server via SSH:

1. **Using SSH Client:** You can use PuTTY, MobaXterm, or a terminal application.
2. **Command to Connect:**

ssh <username>@<server IP address>

(The server IP address will be shared with you via email during account creation.)
3. **Password:** Enter your password when prompted to authenticate.

Note: This server acts as both the master and compute node, so all job submissions and executions happen on the same machine.

Storage Allocation

Once you have logged in, your home directory will be available under /home/<username>. Each user is allocated 200 GB of storage for all purposes, including scripts, datasets, and output files. Users are responsible for managing their data efficiently.

Software and Modules

- **Software Availability:**
  Software such as Anaconda and Python are available in /apps/software.
- **Module Files:**
  Module files for ROCm, Python, and other software are available in /apps/modulefiles.

- **Important Note:**
  The server uses **AMD GPUs (MI300X)**. Do not attempt to run NVIDIA-specific CUDA code. To use NVIDIA code, you must first **hipify** it to ROCm-compatible format.

HIPIFY Documentation:
For detailed instructions on converting CUDA code to HIP, refer to the official HIPIFY documentation:

HIPIFY Documentation Click here

How to Submit a Job

A **SLURM script** defines the requirements for your job, including:

- The number of nodes required.
- The number of CPU cores and amount of memory needed.
- Time limits specifying how long the job is allowed to run.
- Any specific hardware requirements, such as the need for GPUs.

These details allow SLURM to allocate resources efficiently and manage job execution.

An example of a simple SLURM script is given below.

```
#!/bin/bash
#SBATCH --job-name=gpu_docker_job
#SBATCH --output=gpu_docker_job_%j.log
#SBATCH --error=gpu_docker_job_%j.err
#SBATCH --time=04:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32GB
#SBATCH --partition=debug
#SBATCH --gres=gpu:2          #Adjust GPU count as per your user category limits


#Load necessary modules

module load python/3.11
module load rocm/6.2.4        #Example: Load ROCm module for AMD GPU
```

#Activate virtual environment if needed
#source /path/to/your/virtualenv/bin/activate

#Navigate to your script directory
cd /path/to/your/scripts

#Run your container with SLURM-managed GPUs

```
srun --container-image=my_docker_image.sif \
    --container-mounts=/mnt/data:/mnt/data \

    --container-workdir=/mnt/data/$USER \
    python your_script.py
```

Note:
Always ensure the value passed to --gres=gpu:<N> follows the GPU limits defined for your user category.

Job Scheduling and Resource Enforcement

The RBCCPS server uses **SLURM** to strictly control CPU and GPU usage and to ensure fair sharing of resources among users.

The server consists of **8 AMD Instinct MI300X GPUs in total**.

User-wise Resource Limits

PhD Users

- Maximum GPUs per job: **5**
- Maximum running jobs at a time: **2**
- Maximum job submissions at a time: **4**
    - Up to **2 jobs running**
    - Remaining jobs stay in **PENDING (queue)** state

Non-PhD Users (M.Tech / Project / Faculty)

- Maximum GPUs per job: **3**
- Maximum running jobs at a time: **1**
- Maximum job submissions at a time: **2**
    - **1 job running**
    - **1 job in PENDING (queue)** state

GPU Request Rules

- The cluster has **only 8 GPUs in total**
- Users **must not request more GPUs** than:
    - Their **user category allows**
    - The **physical GPUs available** on the server

If a job requests more GPUs than allowed or available:

- The job **will not fail**
- The job will remain in **PENDING** state
- The job **will not start** until the GPU request is corrected or sufficient resources become available

---

GPU Usage Policy

- Direct execution of GPU workloads outside SLURM (e.g., running Python, PyTorch, ROCm programs directly on login shell) is **strictly prohibited**
- All GPU access is **enforced via SLURM**
- Jobs found running on GPUs outside SLURM may be **terminated without notice**, and user access may be revoked

---

SLURM Commands

Command	Purpose
sbatch <script>	Submit a batch job
srun <command>	Run tasks interactively or in a job script
salloc	Allocate resources for interactive jobs
squeue	Monitor job status
squeue -j <job_id>	Check specific job status
sinfo	View partition and node information
scancel <job_id>	Cancel a job
scontrol show job <job_id>	View detailed job information
kill <PID>	Kill a running process

Job Status Codes

- **PD (Pending):** Job is waiting for resources to be available.
- **R (Running):** Job is currently executing on the server.
- **CG (Completing):** Job is finishing and releasing resources.
- **F (Failed):** Job did not complete successfully.

Important Notes

- Always submit jobs with realistic resource requests to avoid long queue times.
- Use salloc or srun for interactive debugging and testing before submitting large batch jobs.
- Regularly monitor jobs with squeue and check output/error logs for any issues.
- Storage capacity per user is **200 GB only**. Clean up unnecessary files to maintain availability.
- We have one SLURM partition which will run for 24 hours; if your job requires more time, please enable checkpointing.

Commands:

```
#!/bin/bash
#SBATCH --job-name=uako-all
#SBATCH --output=logs/uako_all_%j.log
#SBATCH --error=logs/uako_all_%j.err
#SBATCH --time=23:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=25GB
#SBATCH --partition=debug
#SBATCH --gres=gpu:4

# ============================================================================
# Train ALL systems in parallel, each on its own GPU.
#
# Requests 4 GPUs and pins each system to a separate device via
# CUDA_VISIBLE_DEVICES, avoiding shared-context overhead.
#
# To fall back to shared single-GPU mode (e.g., if 4 GPUs are unavailable):
#   sbatch --gres=gpu:1 scripts/cps-cluster-run-all.sh
# Systems will round-robin over whatever GPUs are allocated.
#
# Usage:
#   sbatch scripts/cps-cluster-run-all.sh
#   sbatch scripts/cps-cluster-run-all.sh 42           # custom seed
# ============================================================================

set -euo pipefail

SEED="${1:-4551}"

# --- Paths ---
PROJECT_DIR="$HOME/robotics/universal-koopman"
VENV_DIR="$PROJECT_DIR/.venv"

# --- Modules ---
module purge
module load python/3.11
module load rocm/6.2.4

# --- Environment ---
source "$VENV_DIR/bin/activate"
cd "$PROJECT_DIR"
mkdir -p logs
```

```
# --- Print info ---
echo "============================================"
echo "Job ID:        $SLURM_JOB_ID"
echo "Training ALL systems in parallel"
echo "Seed:          $SEED"
echo "Node:          $(hostname)"
echo "Date:          $(date)"
NUM_GPUS=$(python -c 'import torch; print(torch.cuda.device_count())' 2>/dev/null ||
echo '0')
echo "GPUs:          $NUM_GPUS"
for i in $(seq 0 $((NUM_GPUS - 1))); do
    echo "  GPU $i:       $(python -c "import torch;
print(torch.cuda.get_device_name($i))" 2>/dev/null || echo 'N/A')"
done
echo "============================================"

SYSTEMS="pendulum cartpole two_link planar_quadrotor quad3d"
PIDS=()

GPU_IDX=0
for sys in $SYSTEMS; do
    GPU=$((GPU_IDX % NUM_GPUS))
    echo "[$(date +%H:%M:%S)] Starting $sys on GPU $GPU ..."
    CUDA_VISIBLE_DEVICES=$GPU python -m universal_koopman.train \
        --system "$sys" \
        --seed "$SEED" \
        > "logs/uako_${SLURM_JOB_ID}_${sys}.log" \
        2> "logs/uako_${SLURM_JOB_ID}_${sys}.err" &
    PIDS+=($!)
    echo "  PID: ${PIDS[-1]}"
    GPU_IDX=$((GPU_IDX + 1))
done

echo ""
echo "All ${#PIDS[@]} systems launched. Waiting..."
echo "============================================"

# --- Wait and report per-system exit status ---
FAILED=0
i=0
for sys in $SYSTEMS; do
    wait "${PIDS[$i]}" && \
        echo "[$(date +%H:%M:%S)] $sys finished successfully." || \
        { echo "[$(date +%H:%M:%S)] $sys FAILED (exit code $?).";
FAILED=$((FAILED+1)); }
    i=$((i+1))
done
```

```
echo ""
echo "============================================="
echo "All done at $(date). Failures: $FAILED/${#PIDS[@]}"
echo "============================================="

exit $FAILED
```